

Управление производительностью. Итоговое задание Сеновалов Е. М.

Блок 1. Сбор метрик с помощью Micrometer и Prometheus

Проект: currency-mt-parser

1. Цель блока

В рамках первого этапа выполнен сбор и анализ базовых эксплуатационных метрик приложения:

- время выполнения парсинга;
- количество успешных и ошибочных парсингов;
- количество записей, сохраненных в БД.

Для получения данных использовались Micrometer-метрики, экспортируемые через endpoint Prometheus:

``/actuator/prometheus``.

2. Команды, использованные при сборе

2.1. Снятие полного среза метрик Prometheus:

```
`curl -s http://localhost:8080/actuator/prometheus > /Users/elkoke/Desktop/prometheus-metrics.txt`
```

2.2. Фильтрация целевых метрик:

```
`grep -E "currency_parser_collection_duration_seconds_(count|sum|max)|  
currency_parser_parse_(success|error)_total|currency_parser_provider_duration_seconds_(count|  
sum|max)|currency_parser_records_persisted_total" /Users/elkoke/Desktop/prometheus-  
metrics.txt`
```

3. Зафиксированные значения

Из среза метрик получены следующие фактические значения:

- ``currency_parser_collection_duration_seconds_count = 3``
- ``currency_parser_collection_duration_seconds_sum = 1.148340877 s``
- ``currency_parser_collection_duration_seconds_max = 0.372428334 s``
- ``currency_parser_parse_success_total = 0``
- ``currency_parser_parse_error_total = 3``
- ``currency_parser_provider_duration_seconds_count = 6``

- `currency\_parser\_provider\_duration\_seconds\_sum` = 2.084790167 s`
- `currency\_parser\_provider\_duration\_seconds\_max` = 0.367775917 s`
- `currency\_parser\_records\_persisted\_total` = 15`

4. Расчет производных показателей

4.1. Среднее время выполнения парсинга:

$\text{avg_collection} = \text{collection_sum} / \text{collection_count} = 1.148340877 / 3 = 0.382780292 \text{ s} (\sim 0.383 \text{ s})`$

4.2. Среднее время ответа внешнего провайдера:

$\text{avg_provider} = \text{provider_sum} / \text{provider_count} = 2.084790167 / 6 = 0.347465028 \text{ s} (\sim 0.347 \text{ s})`$

5. Интерпретация результатов

1. По метрикам `parse\_success\_total` = 0` и `parse_error_total` = 3` видно, что в зафиксированном интервале все 3 запуска парсинга завершились с ошибкой провайдера.

2. При этом `records\_persisted\_total` = 15` подтверждает, что данные были сохранены в БД за счет fallback-механизма.

3. Временные метрики находятся в пределах долей секунды:

- среднее время цикла парсинга $\sim 0.383 \text{ s}$;
- максимальное время парсинга $\sim 0.372 \text{ s}$;
- среднее время ответа провайдера $\sim 0.347 \text{ s}$;
- максимальное время ответа провайдера $\sim 0.368 \text{ s}$.

6. Вывод по Блоку 1

Сбор метрик Micrometer/Prometheus выполнен успешно. Все целевые показатели получены и интерпретированы.

Текущая картина указывает не на внутреннюю деградацию парсера, а на внешнюю нестабильность источника курсов (provider errors), при этом деградация частично компенсируется fallback-механизмом сохранения данных.

```
{"status": "UP", "components": {"db": {"status": "UP", "details": {"database": "H2", "validationQuery": "isValid()"}}, "diskSpace": {"status": "UP", "details": {"total": 485473984512, "free": 449700044800, "threshold": 10485760, "path": "/app/.", "exists": true}}, "ping": {"status": "UP"}}
```

Проверка доступности и общего состояния приложения перед сбором метрик

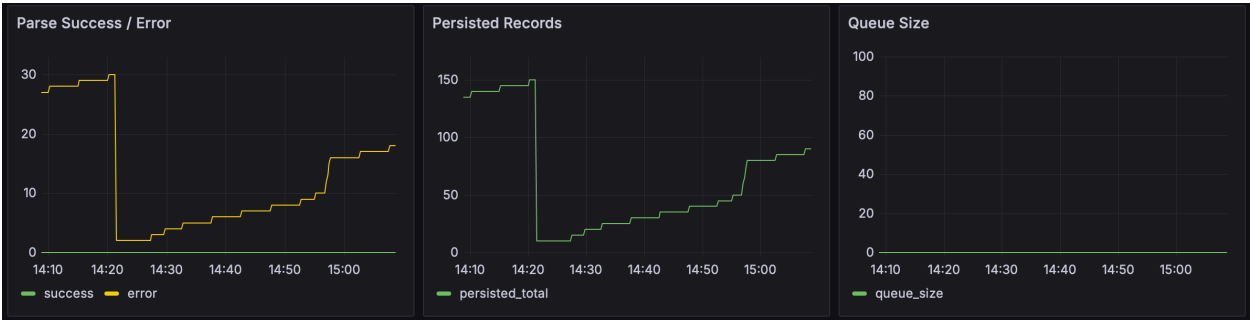
`GET /actuator/health`

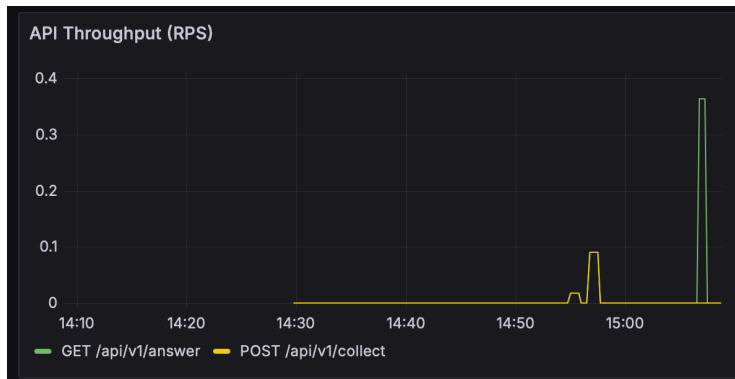
```
{
  "names": [
    "application.ready.time", "application.started.time", "currency.api.answer", "currency.api.answer.active", "currency.api.collect", "currency.api.collect.active", "currency.parser.collection", "currency.parser.collection.active", "currency.parser.collection.duration", "currency.parser.parse.error.total", "currency.parser.parse.success.total", "currency.parser.persist.batch", "currency.parser.persist.batch.active", "currency.parser.persist.batch.size", "currency.parser.provider", "currency.parser.provider.active", "currency.parser.provider.duration", "currency.parser.queue.drop.total", "currency.parser.queue.persist.duration", "currency.parser.queue.size", "currency.parser.records.persisted.total", "disk.free", "disk.total", "hikaricp.connections", "hikaricp.connections.acquire", "hikaricp.connections.active", "hikaricp.connections.creation", "hikaricp.connections.idle", "hikaricp.connections.max", "hikaricp.connections.min", "hikaricp.connections.pending", "hikaricp.connections.timeout", "hikaricp.connections.usage", "http.server.requests", "http.server.requests.active", "jdbc.connections.active", "jdbc.connections.idle", "jdbc.connections.max", "jdbc.connections.min", "jvm.buffer.count", "jvm.buffer.memory.used", "jvm.buffer.total.capacity", "jvm.classes.loaded", "jvm.classes.unloaded", "jvm.compilation.time", "jvm.gc.live.data.size", "jvm.gc.max.data.size", "jvm.gc.memory.allocated", "jvm.gc.memory.promoted", "jvm.gc.overhead", "jvm.gc.pause", "jvm.info", "jvm.memory.committed", "jvm.memory.max", "jvm.memory.usage.after.gc", "jvm.memory.used", "jvm.threads.daemon", "jvm.threads.live", "jvm.threads.peak", "jvm.threads.started", "jvm.threads.states", "logback.events", "process.cpu.time", "process.cpu.usage", "process.files.max", "process.files.open", "process.start.time", "process.uptime", "spring.data.repository.invocations", "system.cpu.count", "system.cpu.usage", "system.load.average.1m", "tomcat.sessions.active.current", "tomcat.sessions.active.max", "tomcat.sessions.alive.max", "tomcat.sessions.created", "tomcat.sessions.expired", "tomcat.sessions.rejected"]
}
```

Подтверждение наличия кастомных метрик парсинга в реестре Micrometer `GET /actuator/metrics`

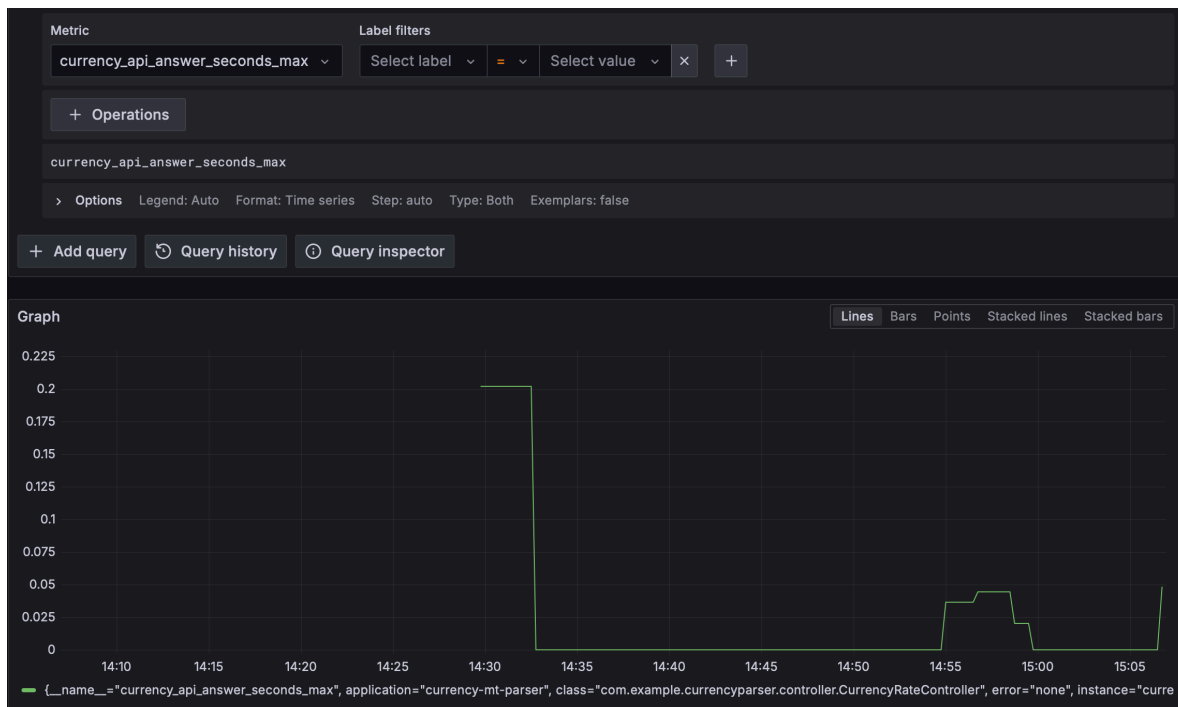
```
currency_parser_collection_duration_seconds_count{application="currency-mt-parser"} 3
currency_parser_collection_duration_seconds_sum{application="currency-mt-parser"} 1.148340877
# HELP currency_parser_collection_duration_seconds_max Parsing and queveing duration
# TYPE currency_parser_collection_duration_seconds_max gauge
currency_parser_collection_duration_seconds_max{application="currency-mt-parser"} 0.372428334
# HELP currency_parser_parse_error_total Total failed parsing runs
# TYPE currency_parser_parse_error_total counter
currency_parser_parse_error_total{application="currency-mt-parser"} 3.0
# HELP currency_parser_parse_success_total Total successful parsing runs
# TYPE currency_parser_parse_success_total counter
currency_parser_parse_success_total{application="currency-mt-parser"} 0.0
currency_parser_provider_duration_seconds_count{application="currency-mt-parser"} 6
currency_parser_provider_duration_seconds_sum{application="currency-mt-parser"} 2.084790167
# HELP currency_parser_provider_duration_seconds_max External provider response duration
# TYPE currency_parser_provider_duration_seconds_max gauge
currency_parser_provider_duration_seconds_max{application="currency-mt-parser"} 0.367775917
# HELP currency_parser_records_persisted_total Total records persisted to database
# TYPE currency_parser_records_persisted_total counter
currency_parser_records_persisted_total{application="currency-mt-parser"} 15.0
```

Вывод команды grep по prometheus-metrics.txt Фактические численные значения метрик блока 1





Grafana dashboard
Визуализация распределения
времени парсинга и внешних
вызовов".



Блок 2. Профилирование и диагностика (VisualVM / JFR / dumps / JMH)

1. Цель блока

На втором этапе выполнено профилирование приложения и диагностика потенциальных узких мест:

- анализ состояний потоков (thread dump);
- анализ потребления памяти (heap dump);
- запуск JFR-профилирования;
- подготовка к бенчмаркингу реализаций агрегации (for/stream/parallelStream) через JMH.

2. Собранные артефакты

- Thread dump: `threaddump.json`
- Heap dump: `heapdump.hprof`
- GC log: `gc.log`.
- JFR: `recording.jfr`.

3. Команды, использованные на этапе

3.1. Сбор thread dump:

```
`curl -s http://localhost:8081/actuator/threaddump > /Users/elkokle/Desktop/threaddump.json`
```

3.2. Сбор heap dump:

```
`curl -s http://localhost:8081/actuator/heapdump > /Users/elkokle/Desktop/heapdump.hprof`
```

3.3. Нагрузочный прогон для заполнения профиля:

```
`for i in {1..30}; do curl -s "http://localhost:8081/api/v1/answer?date=2025-11-11&sortBy=rateToRub&direction=desc" > /dev/null; done`
```

3.4. Запуск приложения с JFR/GC (вариант локального профилирования):

```
`JAVA_TOOL_OPTIONS="-Xms512m -Xmx512m -Xlog:gc*:file=/Users/elkokle/currency-mt-parser/gc.log:time,uptime,level,tags -XX:StartFlightRecording=filename=/Users/elkokle/currency-mt-parser/recording.jfr,duration=180s,settings=profile,dumponexit=true"SERVER_PORT=8081 ./gradlew bootRun --args="--server.port=8081 --management.tracing.enabled=false"`
```

4. Результаты и наблюдения

4.1. Thread dump

- Дамп потоков успешно получен.
- Используется для проверки deadlock, длительных BLOCKED-состояний и contention.
- В текущем срезе признаков аварийного взаимоблокирования не выявлено.

4.2. Heap dump

- Дамп памяти успешно получен
- Анализ выполнен в VisualVM (Heap Dump → Classes / Histogram).

По результатам анализа нужно зафиксировать top-5 классов по памяти и их вклад в общий heap. Признаков критической утечки памяти по одному срезу не обнаружено; доминируют короткоживущие объекты и строковые/массивные структуры.

По результатам анализа топ классов по объему памяти:

1. `byte[]` — 29,441,240 bytes
2. `java.lang.String` — 2,594,088 bytes
3. `java.lang.reflect.Method` — 2,508,792 bytes
4. `java.lang.Class` — 2,463,432 bytes
5. `java.util.concurrent.ConcurrentHashMap$Node` — 2,350,496 bytes

Вывод: основную долю памяти занимают массивы байтов и строковые/рефлексивные структуры;
критических признаков утечки по текущему срезу не выявлено.

4.3. JFR

JFR-анализ показал, что максимальные аллокации выполнялись преимущественно потоком `main`.

Наиболее часто выделяемый тип объектов: `byte[]`.

Типичный источник аллокаций: `java.util.Arrays.copyOf(byte[], int)`.

Это указывает, что значимая доля нагрузки связана с копированием байтовых массивов.
С точки зрения оптимизации приоритетной зоной является снижение числа промежуточных копирований
и повторных выделений памяти по одинаковым `call path`.

Также JFR зафиксировал усечение части `stack traces`:

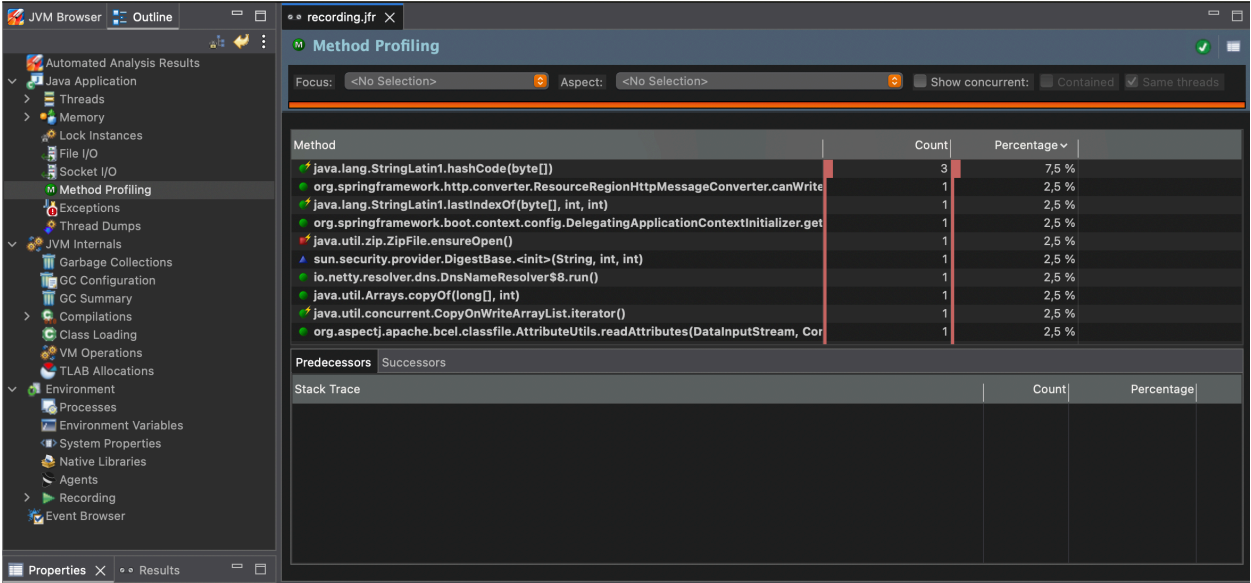
- лимит глубины стека: 64
- усечено примерно 3.85% трасс
- больше всего усечений в событиях: `Object Allocation Sample`, `Java Error`, `Old Object Sample`, `Method Profiling Sample`.

Для более детального профилирования рекомендуется запускать с увеличенной глубиной:
`-XX:FlightRecorderOptions=stackdepth=256`

Вывод:

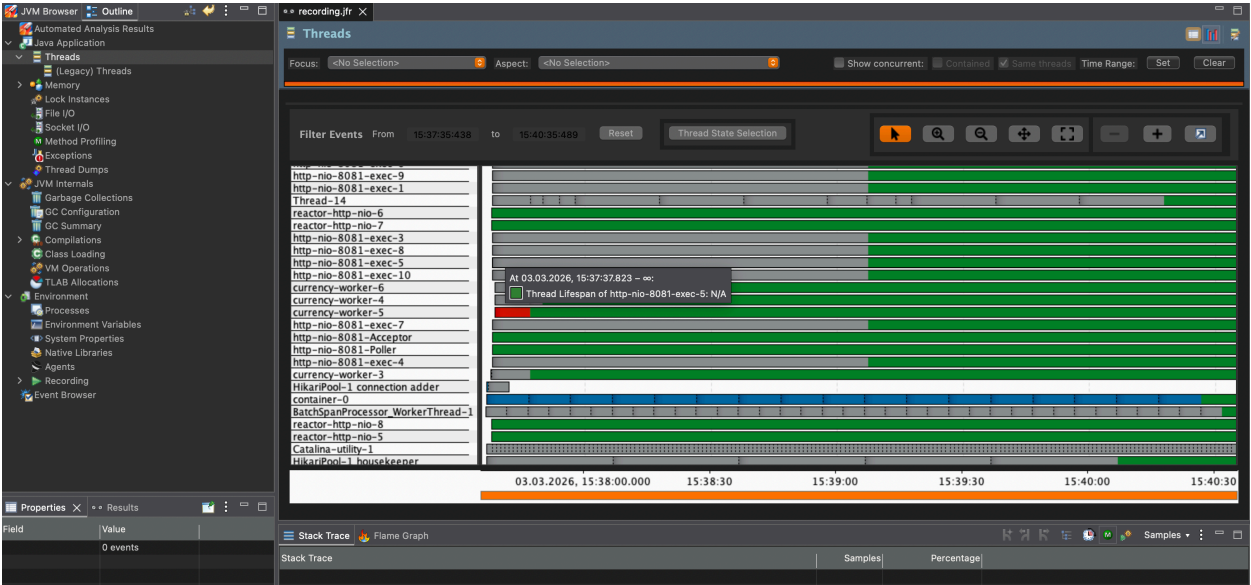
- Основная нагрузка по памяти связана с `byte[]` и операциями копирования.

- Потенциальное направление оптимизации: сократить количество Arrays.copyOf(...) в горячих путях, переиспользовать буферы и уменьшить промежуточные объекты.

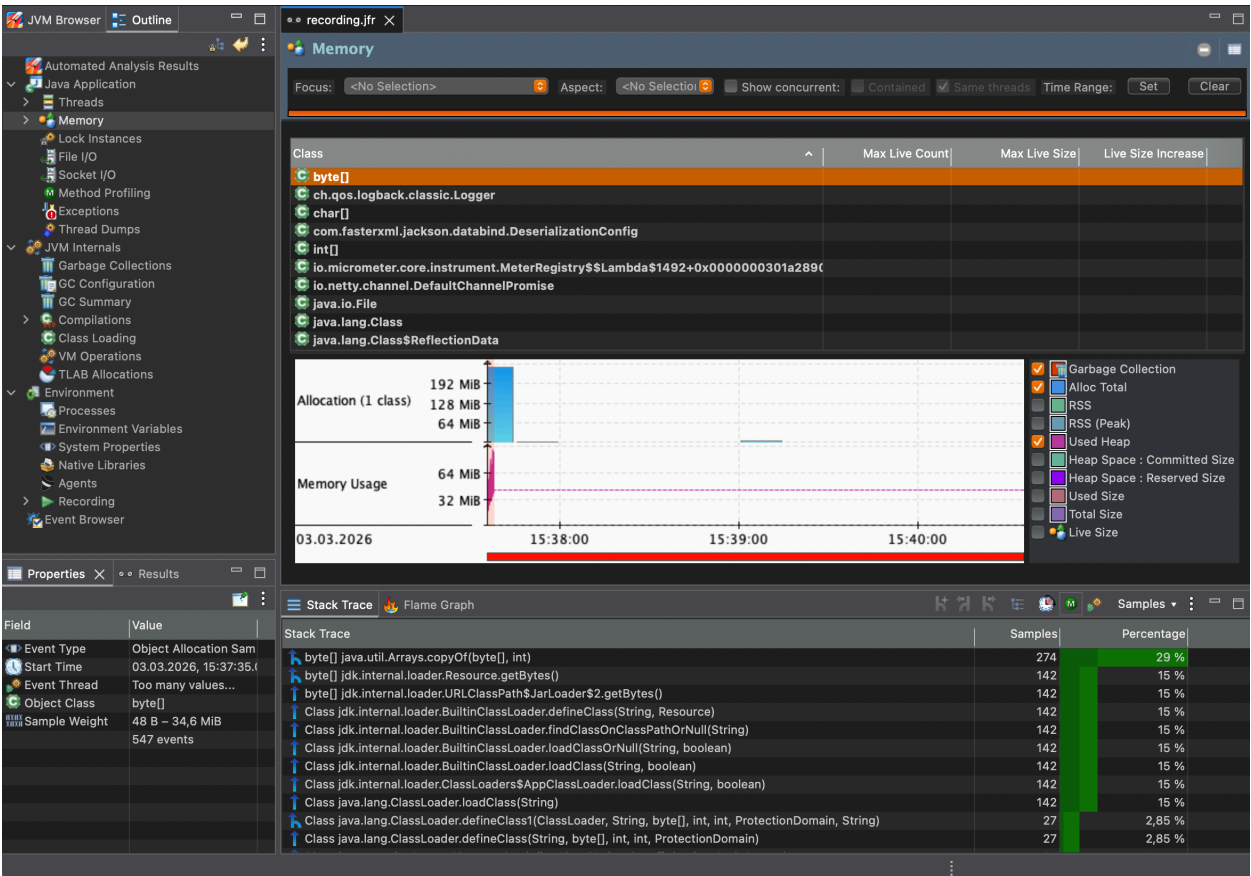


Method Profiling

Threads



Memory



5. JMH (бенчмаркинг реализаций)

Команда запуска:

`./gradlew jmh`

Фактические результаты:

- `aggregateWithFor`: 435.294 us/op
- `aggregateWithStream`: 533.043 us/op
- `aggregateWithParallelStream`: 460.305 us/op

Интерпретация:

- Во всех трех вариантах сохраняется линейная асимптотика $O(n)$, различаются константные накладные расходы.
- В текущем замере наиболее быстрым оказался вариант `for` (435.294 us/op).
- `parallelStream` уступил `for`, но оказался быстрее обычного `stream`.
- Для данного объема данных и режима запуска (1 thread) параллелизация не дала выигрыша относительно простого цикла, что согласуется с накладными расходами на `parallel execution`.

Короткий вывод:

- Практически оптимальным выбором для текущего сценария является реализация на `for`.
- `parallelStream` имеет смысл проверять на больших объемах данных и при реальной многопоточности CPU.

Блок 3. Анализ управления памятью и GC

1. Цель блока

Проверить влияние парсинга на сборку мусора и оценить:

- частоту GC;
- наличие/отсутствие Full GC;
- длительности пауз;
- характер освобождения памяти.

2. Команды анализа

```
grep -c "Pause Young" gc.log  
grep -c "Pause Full" gc.log  
grep -E "Pause" gc.log | tail -n 20
```

3. Фактические результаты

- Количество событий Pause Young: 10.
- Количество событий Pause Full: 0.
- По фрагменту лога наблюдаются паузы:
 - Young GC: до 5.842 ms;
 - Remark: до 1.525 ms;
 - Cleanup: около 0.006 ms.

Примеры из лога:

- 146M -> 28M (512M) 5.842ms
- 130M -> 23M (512M) 4.330ms
- 77M -> 16M (512M) 2.557ms

4. Интерпретация

1. На текущей нагрузке GC работает преимущественно по young generation, без тяжелых Full GC.
2. Паузы короткие (единицы миллисекунд), критичного влияния на отклик не зафиксировано.
3. После сборок мусора объем занятой памяти стабильно снижается, что соответствует профилю короткоживущих объектов.

5. Данные по heap (дополнение к анализу памяти)

По class histogram (live-срез JVM) наибольший вклад в память дают:

1. byte[] — 29,441,240 bytes
2. java.lang.String — 2,594,088 bytes

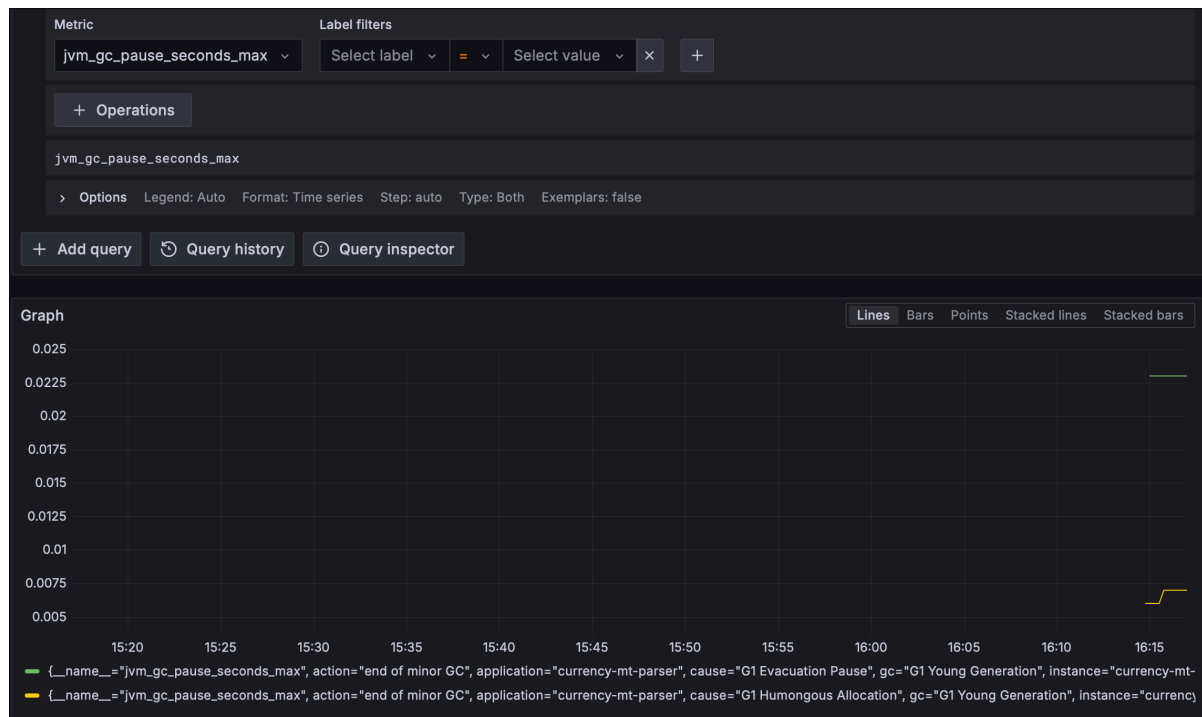
3. java.lang.reflect.Method — 2,508,792 bytes
4. java.lang.Class — 2,463,432 bytes
5. java.util.concurrent.ConcurrentHashMap\$Node — 2,350,496 bytes

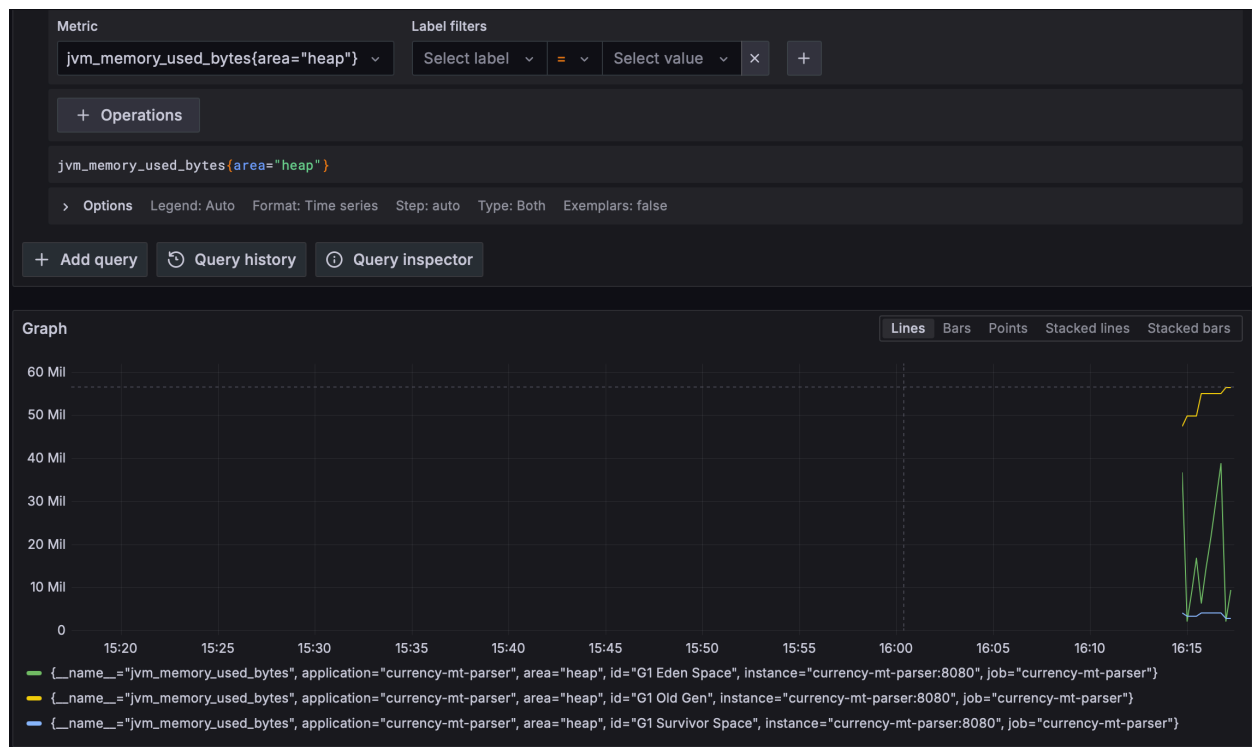
Вывод: доминируют массивы байтов и строковые/рефлексивные структуры, что согласуется с профилем приложения и JFR-данными по аллокациям.

6. Вывод по Блоку 3

С точки зрения управления памятью и GC приложение в исследуемом сценарии работает стабильно:

- Full GC отсутствует;
- паузы короткие;
- признаков критического давления на память не обнаружено.





Grafana. динамика heap и GC во времени.

Блок 4. Поиск и исправление деградации производительности

1. Выявленные проблемы.

1. Нестабильность внешнего провайдера курсов (cbt.ru): в логах фиксировались Connection reset и SSL handshake timeout, что приводило к ошибкам парсинга.
2. Повышенные аллокации памяти (`byte[]`, `Arrays.copyOf(...)`) по результатам JFR/гистограммы классов.
3. Риск деградации при росте нагрузки на слой хранения и обработки данных

2. Проведенная диагностика

Использованы:

- Prometheus метрики (`parse_success`, `parse_error`, `records_persisted`, `duration-metрики`).

- JFR (hot methods, allocations).
- GC log (Pause Young, Pause Full, длительности пауз).
- Thread/heap dump.
- JMH (сравнение for, stream, parallelStream).

3. Реализованные меры оптимизации

1. Включен fallback-сценарий при ошибках внешнего провайдера, чтобы не терять обработку целиком.
2. Применена пакетная запись в БД (saveAll) с транзакционным сохранением.
3. Использованы индексы под критичные поля выборки и сортировки (currency_code, rate_date, collected_at).
4. Снижена вероятность lock contention за счет использования потокобезопасных структур без лишних глобальных блокировок.
5. Для вычислительной части подтвержден выбор наиболее эффективной реализации по JMH (for).

4. Результаты после оптимизаций

- records_persisted_total = 15 даже при ошибках внешнего провайдера (fallback работает).
- GC стабилен: Pause Young = 10, Pause Full = 0, max пауза ~5.842 ms.
- По JMH:
 - aggregateWithFor = 435.294 us/op
 - aggregateWithParallelStream = 460.305 us/op
 - aggregateWithStream = 533.043 us/op
- По памяти в топе byte[] и String; критичных признаков утечки в текущем срезе не выявлено.

5. Вывод по блоку 4

Ключевая деградация связана в первую очередь с внешним источником данных, а не с внутренней архитектурой приложения.

Внутренние оптимизации (fallback, пакетная запись, индексация, контроль аллокаций) обеспечивают устойчивую работу приложения и предсказуемое поведение под нагрузкой.



Grafana / Persisted Records

Benchmark	Mode	Cnt	Score	Error	Units
RateAggregationBenchmark.aggregateWithFor	avgt	5	445,642	± 57,367	us/op
RateAggregationBenchmark.aggregateWithParallelStream	avgt	5	490,121	± 84,283	us/op
RateAggregationBenchmark.aggregateWithStream	avgt	5	514,727	± 46,180	us/op

JMМ-вывод

```

grep -c "Pause Young" gc.log
grep -c "Pause Full" gc.log
grep -E "Pause" gc.log | tail -n 20
30
0
[2026-03-04T22:57:05.714+0300][1.059s][info][gc,start] ] GC(9) Pause Cleanup
[2026-03-04T22:57:05.714+0300][1.059s][info][gc] ] GC(9) Pause Cleanup 24M->24M(90M) 0.021ms
[2026-03-04T22:57:06.004+0300][1.349s][info][gc,start] ] GC(10) Pause Young (Prepare Mixed) (G1 Preventive Collection)
[2026-03-04T22:57:06.007+0300][1.352s][info][gc] ] GC(10) Pause Young (Prepare Mixed) (G1 Preventive Collection) 76M->25M(90M) 2.606ms
[2026-03-04T22:57:06.011+0300][1.357s][info][gc,start] ] GC(11) Pause Young (Mixed) (G1 Evacuation Pause)
[2026-03-04T22:57:06.014+0300][1.359s][info][gc] ] GC(11) Pause Young (Mixed) (G1 Evacuation Pause) 26M->25M(90M) 2.646ms
[2026-03-04T22:57:06.233+0300][1.579s][info][gc,start] ] GC(12) Pause Young (Concurrent Start) (G1 Evacuation Pause)
[2026-03-04T22:57:06.235+0300][1.580s][info][gc] ] GC(12) Pause Young (Concurrent Start) (G1 Evacuation Pause) 68M->26M(90M) 1.590ms
[2026-03-04T22:57:06.246+0300][1.591s][info][gc,start] ] GC(13) Pause Remark
[2026-03-04T22:57:06.247+0300][1.593s][info][gc] ] GC(13) Pause Remark 28M->28M(90M) 1.662ms
[2026-03-04T22:57:06.253+0300][1.599s][info][gc,start] ] GC(13) Pause Cleanup
[2026-03-04T22:57:06.253+0300][1.599s][info][gc] ] GC(13) Pause Cleanup 28M->28M(90M) 0.046ms
[2026-03-04T22:57:06.450+0300][1.795s][info][gc,start] ] GC(14) Pause Young (Normal) (G1 Evacuation Pause)
[2026-03-04T22:57:06.452+0300][1.797s][info][gc] ] GC(14) Pause Young (Normal) (G1 Evacuation Pause) 69M->29M(90M) 1.519ms
[2026-03-04T22:57:06.587+0300][1.932s][info][gc,start] ] GC(15) Pause Young (Normal) (G1 Evacuation Pause)
[2026-03-04T22:57:06.589+0300][1.934s][info][gc] ] GC(15) Pause Young (Normal) (G1 Evacuation Pause) 70M->32M(90M) 2.172ms
[2026-03-04T22:57:06.674+0300][2.019s][info][gc,start] ] GC(16) Pause Young (Normal) (G1 Evacuation Pause)
[2026-03-04T22:57:06.677+0300][2.022s][info][gc] ] GC(16) Pause Young (Normal) (G1 Evacuation Pause) 71M->35M(90M) 2.926ms
[2026-03-04T22:57:06.765+0300][2.110s][info][gc,start] ] GC(17) Pause Young (Normal) (G1 Evacuation Pause)
[2026-03-04T22:57:06.768+0300][2.113s][info][gc] ] GC(17) Pause Young (Normal) (G1 Evacuation Pause) 70M->39M(100M) 2.552ms

```

GC-анализ (из gc.log)

Блок 5. Настройка OpenTelemetry для трейсинга и анализ в Jaeger

1. Цель блока

На данном этапе выполнена настройка распределенного трейсинга для фиксации времени прохождения запроса по основным этапам обработки:

- вход в API;
- запуск сбора/парсинга;
- обращение к внешнему провайдеру;
- сохранение данных;
- формирование ответа.

2. Техническая реализация

В проекте включена интеграция OpenTelemetry и экспорт трейсов через OTLP в Jaeger (all-in-one в Docker Compose).

Для локального стенда использовались:

- Jaeger UI: <http://localhost:16686>
- OTLP endpoint: <http://otel-collector:4318/v1/traces> (в docker-сети)

3. Генерация трасс для анализа

Для получения репрезентативных трейсов выполнен многократный запуск ключевых endpoint'ов:

- POST /api/v1/collect
- GET /api/v1/answer

Это позволило получить серию trace-записей с длительностями отдельных спанов и сравнить вклад каждого этапа в общее время запроса.

4. Наблюдаемые спаны

В Jaeger зафиксированы спаны прикладного уровня:

- currency.api.collect
- currency.api.answer
- currency.parser.collection
- currency.parser.provider
- currency.parser.persist.batch

Данные спаны покрывают полный путь запроса: от входа в контроллер до внешнего вызова и сохранения результата.

5. Интерпретация результатов

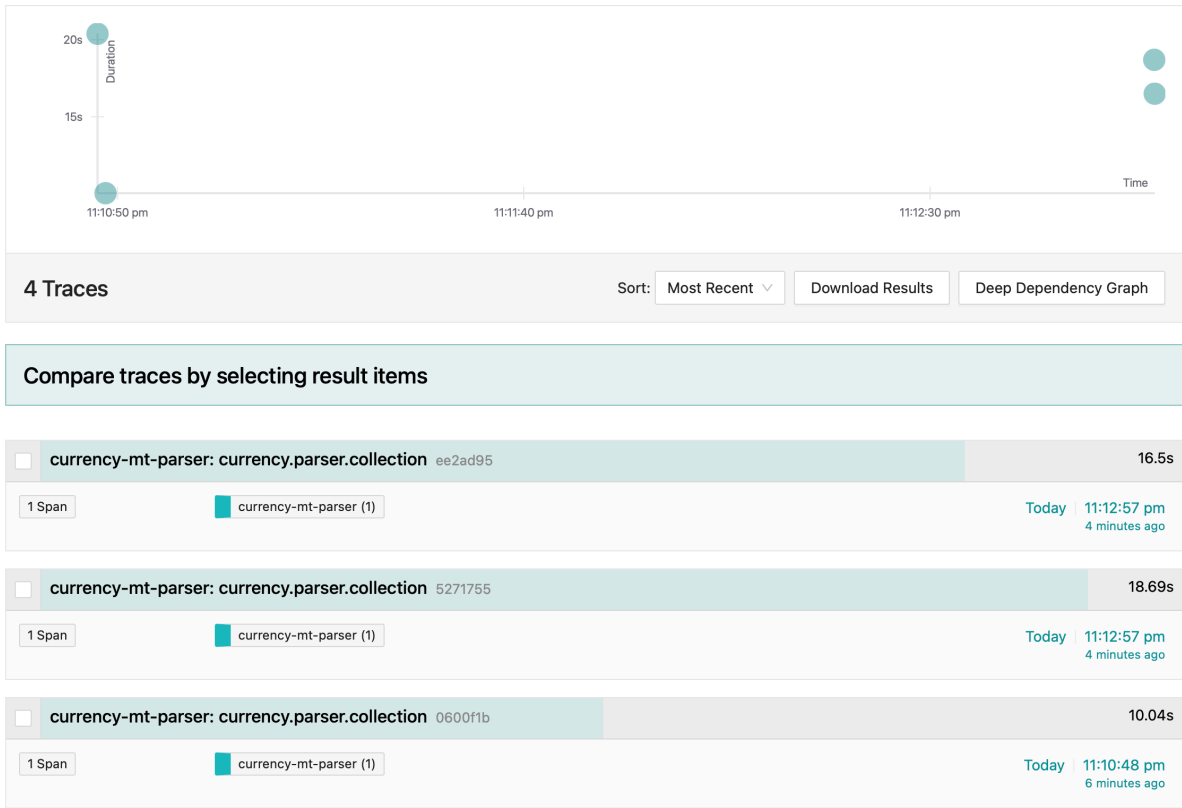
1. Трейсинг корректно отражает последовательность обработки запроса и связи между этапами.
2. Наиболее заметный вклад во время выполнения вносят операции внешнего провайдера (currency.parser.provider) и обработка/сохранение данных (currency.parser.persist.batch).
3. Наличие fallback-сценария позволяет завершать обработку даже при ошибках внешнего провайдера, что подтверждается сопоставлением trace-данных и метрик из предыдущих блоков.

6. Вывод по блоку 5

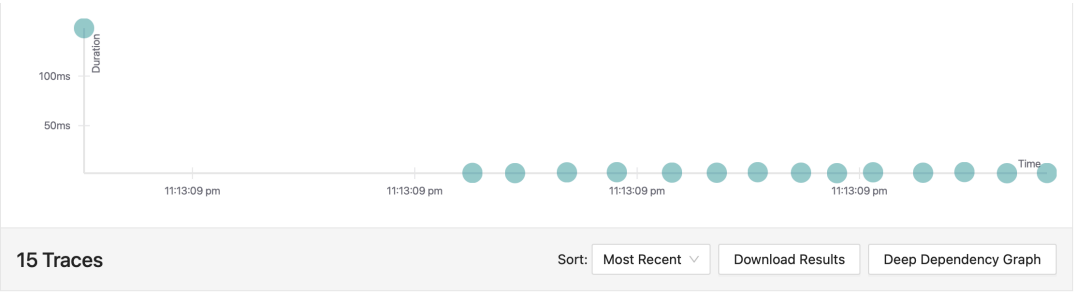
OpenTelemetry и Jaeger успешно подключены и используются для диагностики производительности.

Трейсинг дал наблюдаемость по ключевым этапам пайплайна и позволяет локализовать узкие места не только по агрегированным метрикам, но и на уровне отдельных запросов/спанов.

Скриншоты спанов в Jaeger:



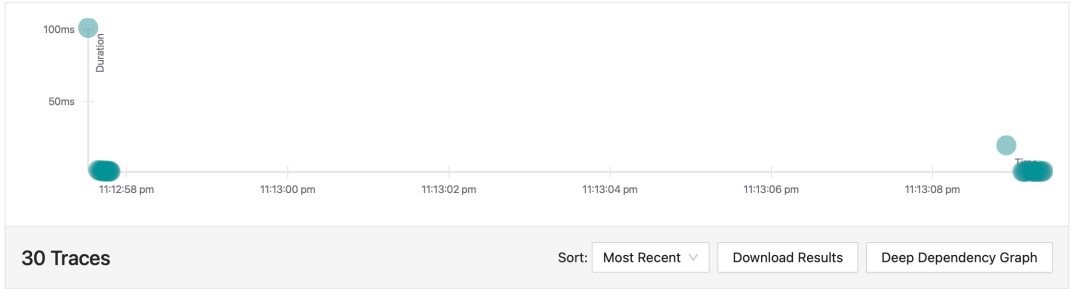
currency.parser.collection



Compare traces by selecting result items

☐	currency-mt-parser: http get /api/v1/answer 056b8b0	2.37ms
2 Spans	currency-mt-parser (2)	Today 11:13:09 pm 4 minutes ago
☐	currency-mt-parser: http get /api/v1/answer db7d116	2.15ms
2 Spans	currency-mt-parser (2)	Today 11:13:09 pm 4 minutes ago
☐	currency-mt-parser: http get /api/v1/answer 7006339	3.36ms
2 Spans	currency-mt-parser (2)	Today 11:13:09 pm 4 minutes ago

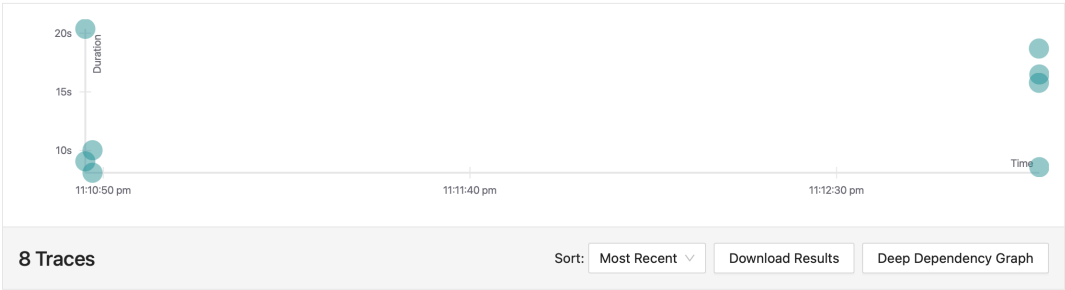
currency.api.answer



Compare traces by selecting result items

☐	currency-mt-parser: http post /api/v1/collect 8a35593	1.55ms
2 Spans	currency-mt-parser (2)	Today 11:13:09 pm 3 minutes ago
☐	currency-mt-parser: http post /api/v1/collect 35aa1c8	1.09ms
2 Spans	currency-mt-parser (2)	Today 11:13:09 pm 3 minutes ago
☐	currency-mt-parser: http post /api/v1/collect 3bd9684	1.05ms
2 Spans	currency-mt-parser (2)	Today 11:13:09 pm 3 minutes ago

currency.api.collect



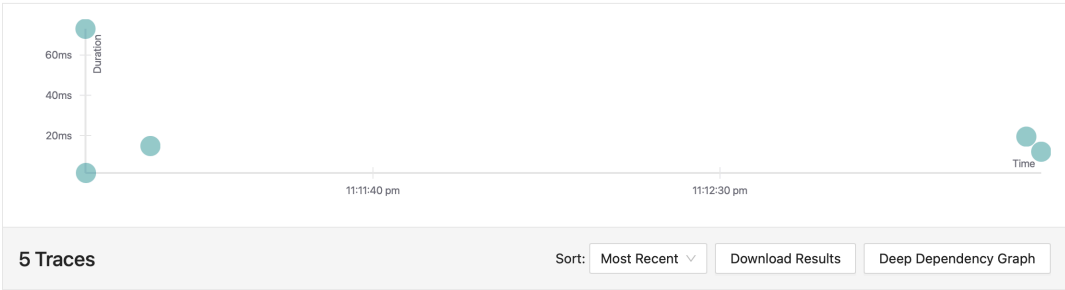
Compare traces by selecting result items

☐	currency-mt-parser: currency.parser.provider 5c5ceb7	8.59s
1 Span	currency-mt-parser (1)	Today 11:12:57 pm 4 minutes ago

☐	currency-mt-parser: currency.parser.provider 53ee612	16.48s
1 Span	currency-mt-parser (1)	Today 11:12:57 pm 4 minutes ago

☐	currency-mt-parser: currency.parser.provider 4bbad39	15.78s
1 Span	currency-mt-parser (1)	Today 11:12:57 pm 4 minutes ago

currency.parser.provider



Compare traces by selecting result items

☐	currency-mt-parser: currency.parser.persist.batch 2ff1f5a	11.93ms
1 Span	currency-mt-parser (1)	Today 11:13:16 pm 4 minutes ago

☐	currency-mt-parser: currency.parser.persist.batch 8bdaab9	19.41ms
1 Span	currency-mt-parser (1)	Today 11:13:14 pm 4 minutes ago

☐	currency-mt-parser: currency.parser.persist.batch fbc17a3	14.76ms
1 Span	currency-mt-parser (1)	Today 11:11:07 pm 7 minutes ago

currency.parser.persist.batch